

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Regression for Prediction

Overfitting, regularisation, and the workflow that finds the sweet spot

Where We Are

01

Our problem

Simple linear models can **underfit** (high bias), like the "U-shape" in the `mpg` ~ `horsepower` residuals.

03

The new problem

High-capacity models easily **overfit** (high variance), memorising the noise in the training data.

We have seen the *theory* of λ . Now we need a practical, robust workflow to actually **find** the best λ and build the best predictive model — which means switching from `statsmodels`, built for inference, to `scikit-learn`, built for prediction.

02

A solution

Polynomial regression adds non-linear features ($X^2, X^3, \dots$), giving the model higher **capacity** to fit the curve.

04

A better solution

Regularisation (Ridge L_2 or Lasso L_1) adds a penalty λ that controls capacity, finding the sweet spot in the bias-variance trade-off.



The Interpretability Trade-Off

What is the "effect" of horsepower on MPG?

Linear Model

Simple answer: β_1 .

One number tells the whole story. Easy to explain and understand.

This is the pivot point. If we're willing to sacrifice simple interpretability for better fit, why stop here? **This is where we cross from statistical inference into Machine Learning.**

Quadratic Model

The effect *depends on the value of horsepower*. No single number captures it.



Shifting Perspectives: From Understanding to Prediction

Statistical Approach

- Focus on p-values and R^2
- Interpret coefficients
- Understand relationships
- Use training data metrics

Machine Learning Approach

- Focus on predictive accuracy
- Optimize for unseen data
- Embrace complexity
- Use test set metrics

When prediction is the goal, we only care about performance on **new, unseen data** (the test set). This requires new metrics and a new mindset.



Metrics for Prediction

1

Mean Squared Error (MSE)

$$MSE_{test} = \frac{1}{m} \sum_{j=1}^m (y_j - \hat{y}_j)^2$$

Average of squared errors. Problem: units are squared (e.g., metres²), making interpretation difficult.

2

Root Mean Squared Error (RMSE)

$$RMSE_{test} = \sqrt{MSE_{test}}$$

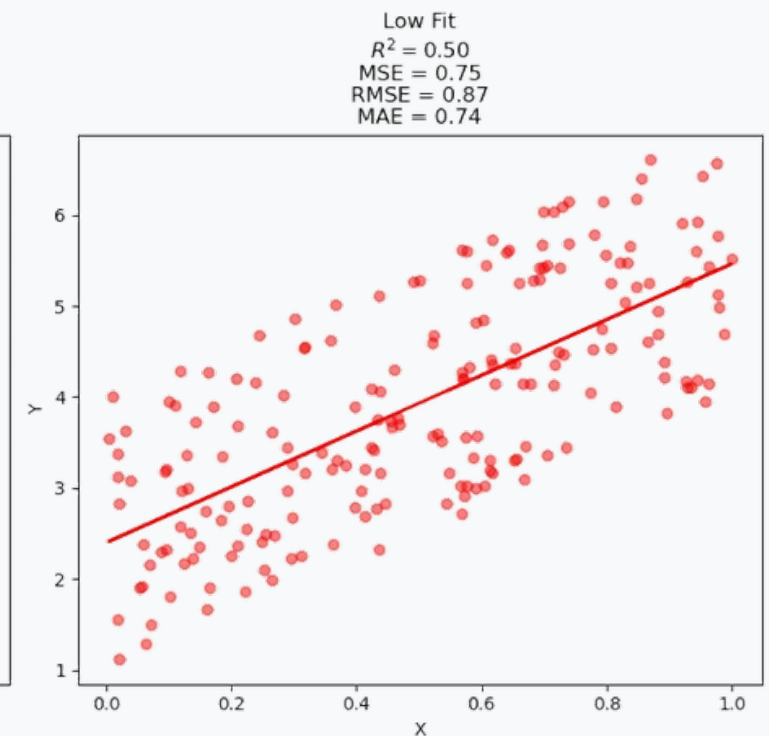
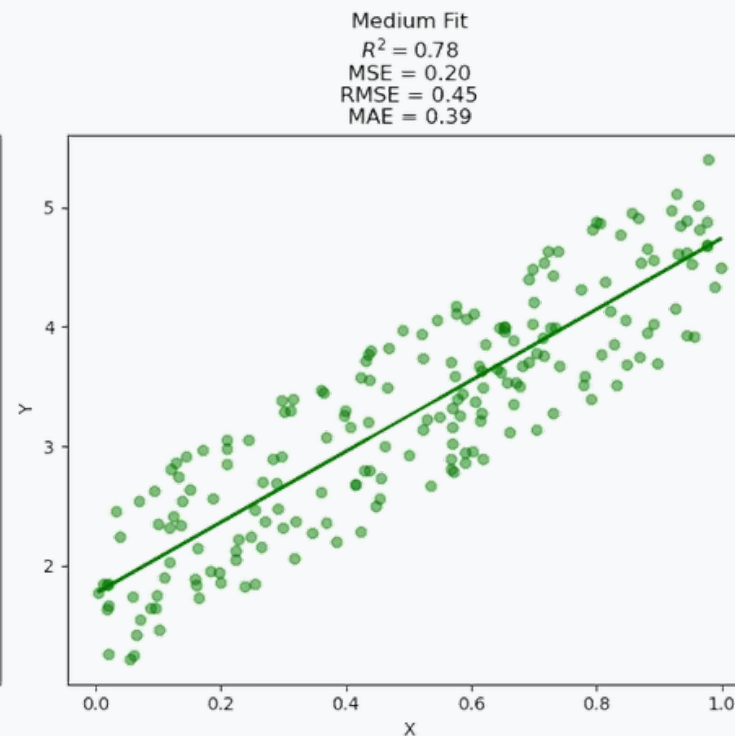
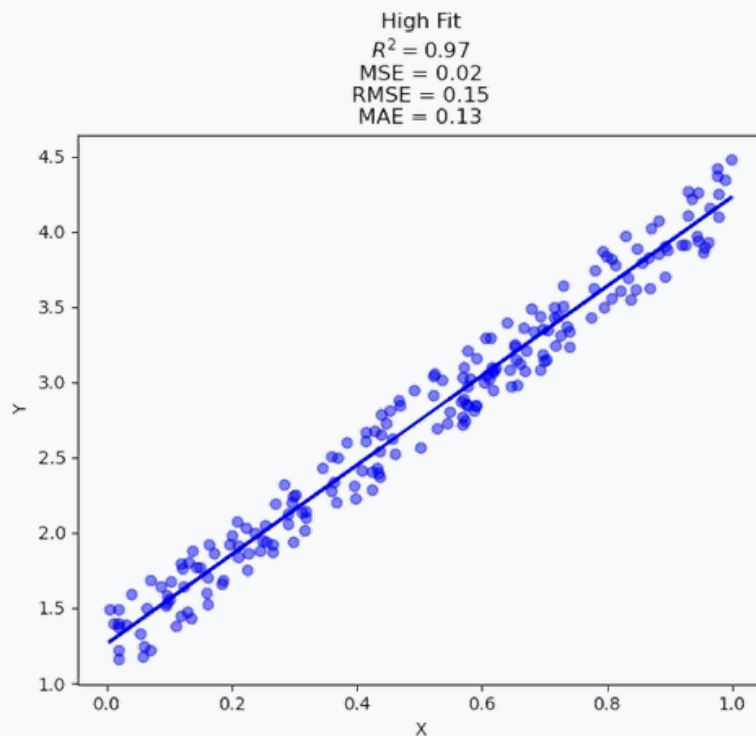
Most common metric. Same units as Y. Penalises large errors more heavily. "On average, predictions are wrong by [RMSE]."

3

Mean Absolute Error (MAE)

$$MAE_{test} = \frac{1}{m} \sum_{j=1}^m |y_j - \hat{y}_j|$$

Average absolute error. Same units as Y. More robust to outliers than RMSE — doesn't disproportionately penalise large errors.



The Overfitting Danger

If degree=2 is good, is degree=20 better?

Underfitting

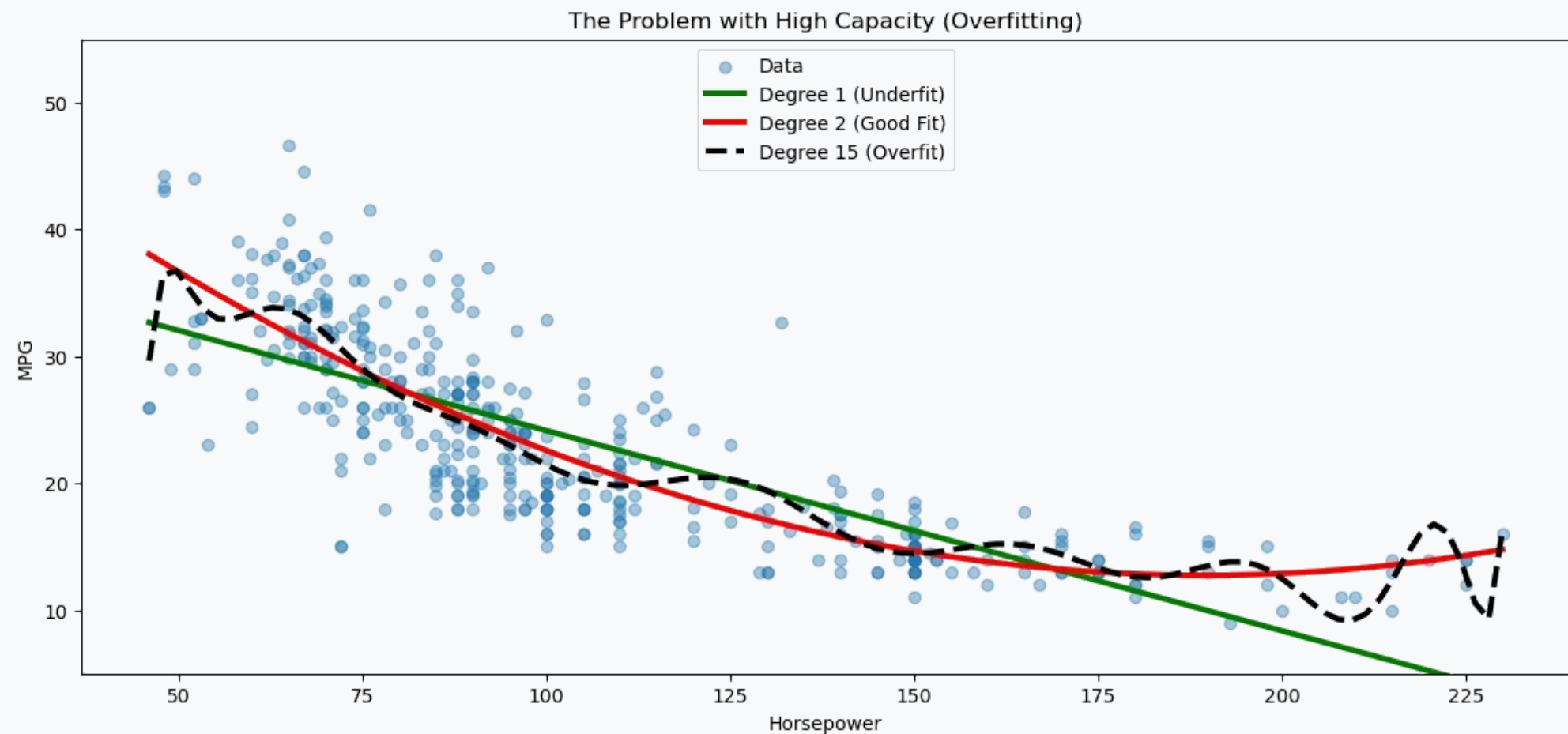
Linear model is too simple. Misses the pattern. **High bias.**

Just Right

Quadratic model captures the signal. Balanced bias-variance.

Overfitting

High-degree polynomial learns the noise. **High variance.**



- A high-degree polynomial can "wobble" to fit every single training point, achieving perfect R^2 on training data but failing miserably on new data.

The Solution: Regularisation

Regularisation adds a penalty to the cost function to discourage model complexity:

$$RSS + \text{Penalty Term}$$

1

RSS Term

Pushes the model to fit the data (reduces bias)

2

Penalty Term

Forces coefficients to stay small (reduces variance)

3

Balance

Find the sweet spot between bias and variance

A "wiggly" overfit curve requires massive β coefficients. By penalising large coefficients, we force the model to find a simpler, smoother curve.

- **Bonus:** Regularisation also solves multicollinearity by stabilising the coefficient estimates.



Ridge Regression (L_2 Penalty)

Ridge adds a penalty proportional to the **sum of squares** of coefficients:

$$RSS_{L2} = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n \beta_j^2$$

This is solved with OLS as follows:

$$\hat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T y$$

where the λI term improves conditioning (invertibility) of the matrix even in the presence of multicollinearity.

$\lambda = 0$

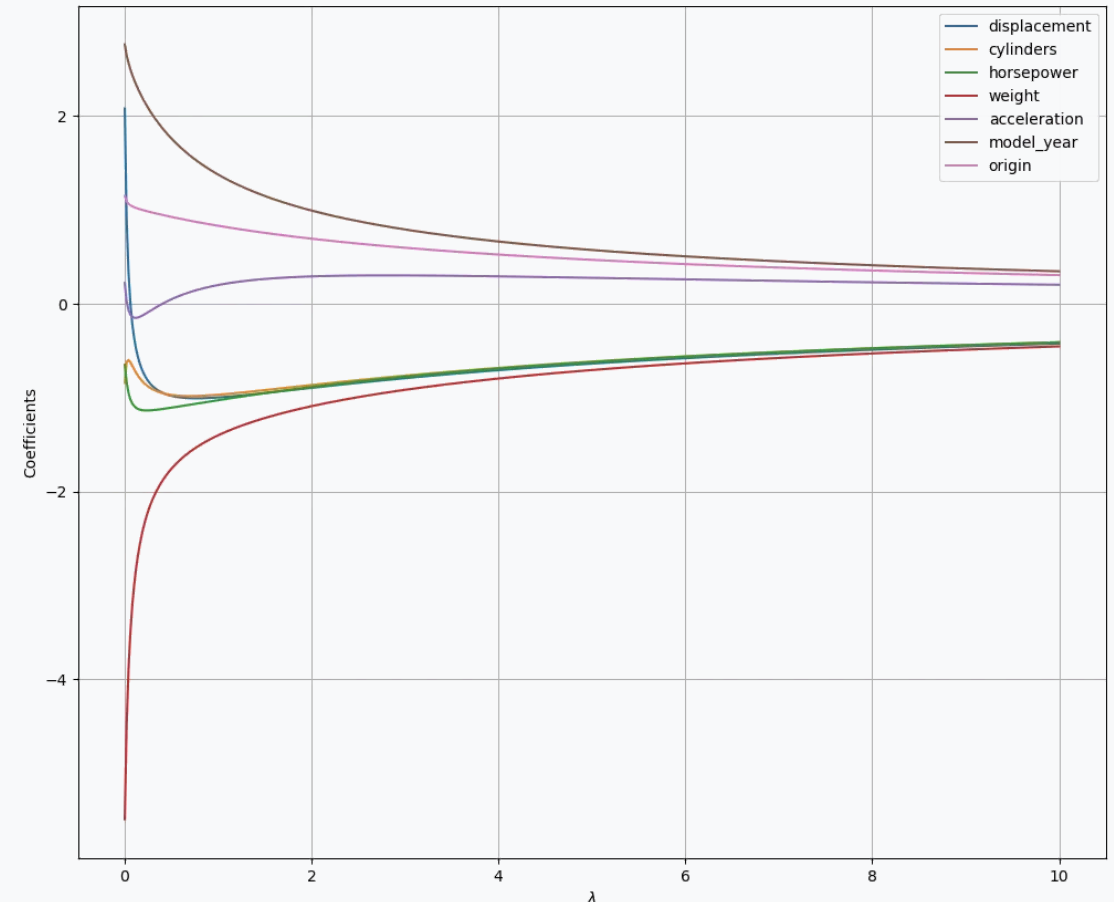
Standard OLS. No penalty.
Risk of overfitting.

$\lambda = \text{optimal}$

Sweet spot. Balanced bias-variance trade-off.

$\lambda \rightarrow \infty$

All coefficients forced to zero. Flat line. Underfitting.



- **Critical:** Standardise (z-score) all features first. Otherwise features with different scales will be penalised unfairly.

Interpreting Ridge Coefficients

Variable	Ridge	OLS
displacement	−0.998	2.079
cylinders	−0.970	−0.841
horsepower	−1.027	−0.652
weight	−1.402	−5.492
acceleration	0.200	0.222
model_year	1.376	2.762
origin	0.830	1.147

- Ridge parameters have smaller magnitude due to regularisation. They **can't be interpreted statistically** like OLS coefficients. Instead, they indicate **relative importance** of variables for prediction.



Lasso Regression (L_1 Penalty)

Lasso uses the **sum of absolute values** of coefficients:

$$RSS_{L1} = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |\beta_j|$$

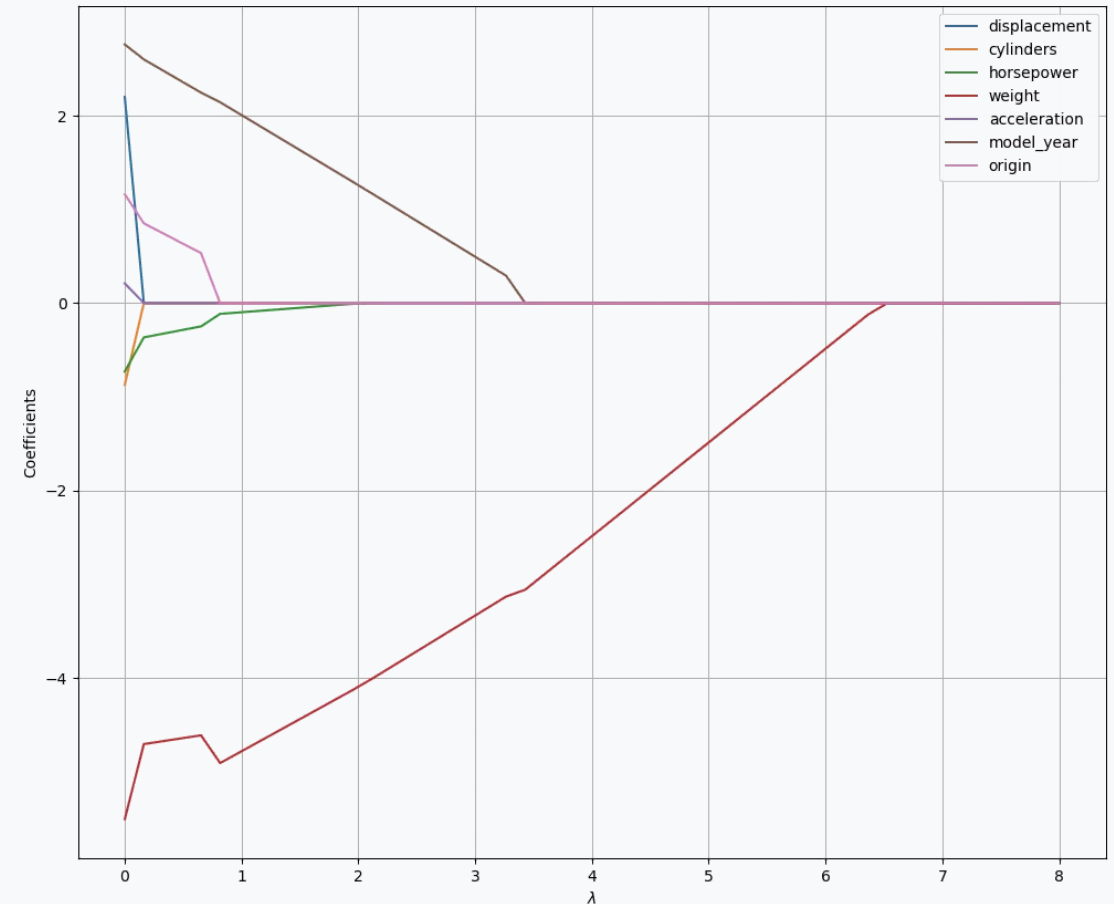
Ridge vs. Lasso

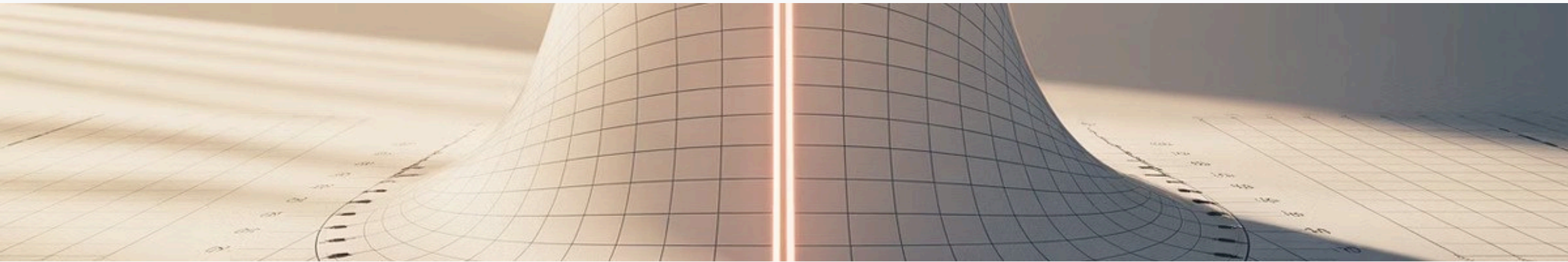
- **Ridge:** Shrinks coefficients smoothly
- **Lasso:** Sets coefficients **exactly to zero**

The "sharp corners" of the absolute value function allow Lasso to eliminate features entirely.

Lasso's Characteristic

Automatic feature selection. When you have hundreds of features, Lasso identifies which ones matter and removes the rest. This creates simpler, more interpretable models.





The Bias-Variance Trade-Off: λ Is the Knob

$\lambda = 0$ (no penalty)

Low bias, high variance

Overfitting: perfect fit on training data, poor on test data

$\lambda = \text{optimal}$

Balanced

Sweet spot: lowest total error on test data

$\lambda \rightarrow \infty$ (heavy penalty)

High bias, low variance

Underfitting: flat line, poor fit everywhere

λ is our "**tradeoff knob**" — the entire goal of machine learning is finding the optimal λ that minimises test error.



Validation Curve: Finding the Sweet Spot

Left side ($\lambda \approx 0.001$)

Training error very low, test error high.
Overfitting (high variance region).

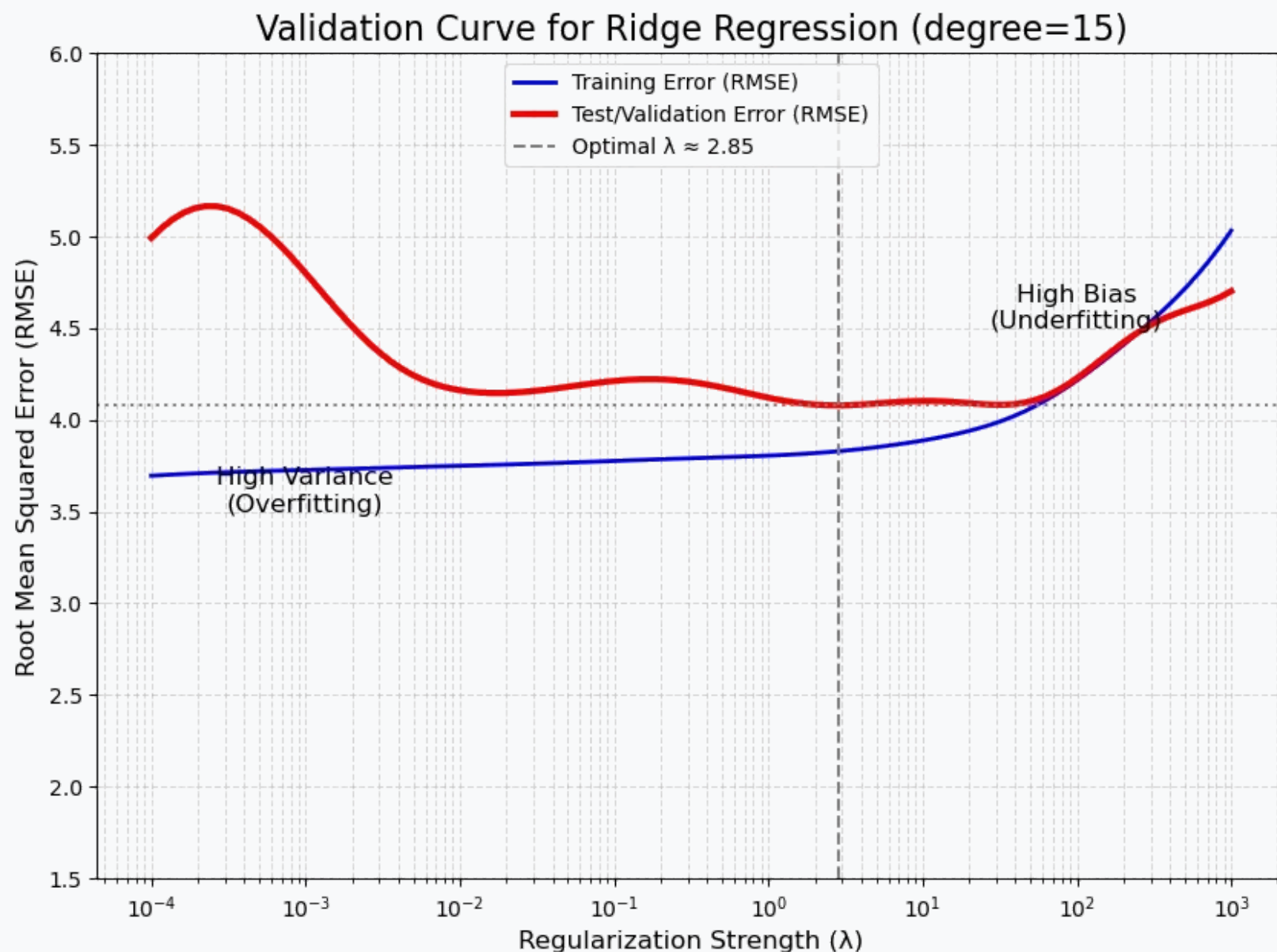
Sweet spot (dashed line)

Test error at minimum. **Optimal λ** — perfect balance between bias and variance.

Right side ($\lambda > 1000$)

Both errors high and similar. **Underfitting** (high bias region).

This shows how regularisation controls the bias-variance trade-off for a degree-15 polynomial. Cross-validation helps us find the optimal λ systematically.



From Theory to Practice: Our New Toolkit

The workflow requires new rules and tools:

1

Train/test split

The **Golden Rule**. We *must* split the data to simulate a "future" test set. Our only goal is the best score on it.

2

Standardisation

We *must* scale the features (`StandardScaler`) so that the regularisation penalty λ affects them all fairly.

3

Pipelines

Bundle the steps — scaling, polynomial features, model — together. This prevents **data leakage** and makes the workflow reproducible.



`statsmodels` was built for **inference**: p-values, confidence intervals, interpretable summaries.
`scikit-learn` is built for **prediction**: one interface for every algorithm, and everything needed to measure performance on data the model has not seen.



The Dataset: California Housing

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
X = data.data      # the features
y = data.target    # the targets
print(X.shape, y.shape)
```

```
(20640, 8) (20640,)
```

20 640 suburbs, 8 numerical variables. The target is the median house value of the district; the predictors are median income, house age, average rooms and bedrooms, population, occupancy, latitude and longitude.

For this part we treat the data as a plain matrix of numbers: scikit-learn can load it that way directly.



Data Splitting

We split into a **training**, a **validation** and a **test set** — 60 : 20 : 20 — with `train_test_split`, in two steps.

```
from sklearn.model_selection import train_test_split

X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.2/(1-0.2), random_state=42)
```

```
(12384, 8) (4128, 8) (4128, 8)
0.6 0.2 0.2
```

The split is random; `random_state` is passed so the result can be reproduced. The function keeps the i -th row of `y` aligned with the i -th row of `X`.

- We now reason **on the validation set**, comparing models and parameter configurations. The test set is touched once, at the end.



Data Normalisation, and Where Leakage Comes From

We z-score the features. Note *when*: the scaler is fitted **on the training set only**, so that not even a mean or a standard deviation is computed from the validation or test data.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)           # tunes the parameters
X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

Every scikit-learn object shares one interface:

- **fit** tunes the internal parameters — means and standard deviations here, weights for a regressor
- **transform** transforms the data without tuning anything, so it is safe on validation and test
- **predict** returns predictions
- **score** returns a standard performance measure

- This may look like a trivial detail. It is the most common way a predictive result turns out to be **too good to be true**: a model evaluated on data whose statistics it has already seen.



The Linear Regressor

```
from sklearn.linear_model import LinearRegression

linear_regressor = LinearRegression()
linear_regressor.fit(X_train, y_train)

y_val_pred = linear_regressor.predict(X_val)
```

```
coefficients:
[ 0.860  0.120 -0.280  0.312 -0.010
 -0.026 -0.888 -0.862]
intercept: 2.068

MAE 0.533    MSE 0.530    RMSE 0.728
```

The same `fit` / `predict` pair will now serve every model we try. **The score on the validation set is the only thing that will decide between them** — and RMSE 0.728 is the number the rest of the lecture has to beat.



Non-Linear Regression: Sweeping the Degree

Adding polynomial features raises capacity. Watch what it does to the two errors:

Degree	Train RMSE	Validation RMSE
1	0.72	0.73
2	0.65	0.95
3	0.58	1 449.92

- Degree 3 is not a typo. The training error keeps falling — 0.72, 0.65, 0.58 — while the validation error explodes. This is overfitting caught in the act, on real data, in three lines of code.



Ridge Regularisation: Sweeping α

Degree 2 overfits. Ridge lets us keep the capacity and control it. In scikit-learn the penalty is called α :

α	Train RMSE	Validation RMSE
0	0.65	0.96
100	0.66	0.74
200	0.67	0.72
300	0.68	0.71
400	0.68	0.71

As α increases the training error rises and the validation error falls — exactly the trade the penalty is making. At $\alpha = 300$ the polynomial ridge model finally beats the plain linear regressor (0.71 against 0.73).

Lasso, tried the same way on this data, does not improve on it.



Grid Search

Polynomial degree and α are both hyperparameters, and we have been tuning them by hand. In practice we define a **grid** of values, train and validate a model for each combination, and keep the best.

```
def grid_search(alphas=range(200, 400, 25), degrees=range(5)):
    ...
```

```
Evaluating a=200 d=0 MSE=1.37
Evaluating a=200 d=1 MSE=0.53
Evaluating a=200 d=2 MSE=0.51
Evaluating a=200 d=3 MSE=8161.95
Evaluating a=200 d=4 MSE=10884349.27
...
```

Testing the whole range, the best result is **degree 2 with $\alpha = 300$** — the combination we had found by hand, now found systematically.

The failures are as informative as the winner: degree 4 with a light penalty produces an error of ten million.



Pipelines, and Grid Search with Cross-Validation

A predictive model is usually a **stack** of components: standardisation, polynomial expansion, ridge regression. A **Pipeline** bundles them so the whole stack is fitted, tuned and validated as one object — and so the scaler is never fitted on validation data.

```
polynomial_regressor = Pipeline([
    ('scaler', StandardScaler()),
    ('polynomial_expansion', PolynomialFeatures()),
    ('ridge_regression', Ridge()),
])

gs = GridSearchCV(
    polynomial_regressor,
    param_grid={'polynomial_expansion__degree': range(0, 5),
                'ridge_regression__alpha': range(200, 400, 25)},
    scoring=make_scorer(mean_squared_error, greater_is_better=False))
gs.fit(X_trainval, y_trainval)
gs.best_params_
```

```
{'polynomial_expansion__degree': 2,
 'ridge_regression__alpha': 250}
```

With **GridSearchCV**, each combination is scored by **K-fold cross-validation** instead of one fixed validation set — which is what you want when a separate validation set is a luxury you cannot afford.

The refitted model scores **RMSE 0.712** on the validation set.



Comparison and Model Selection

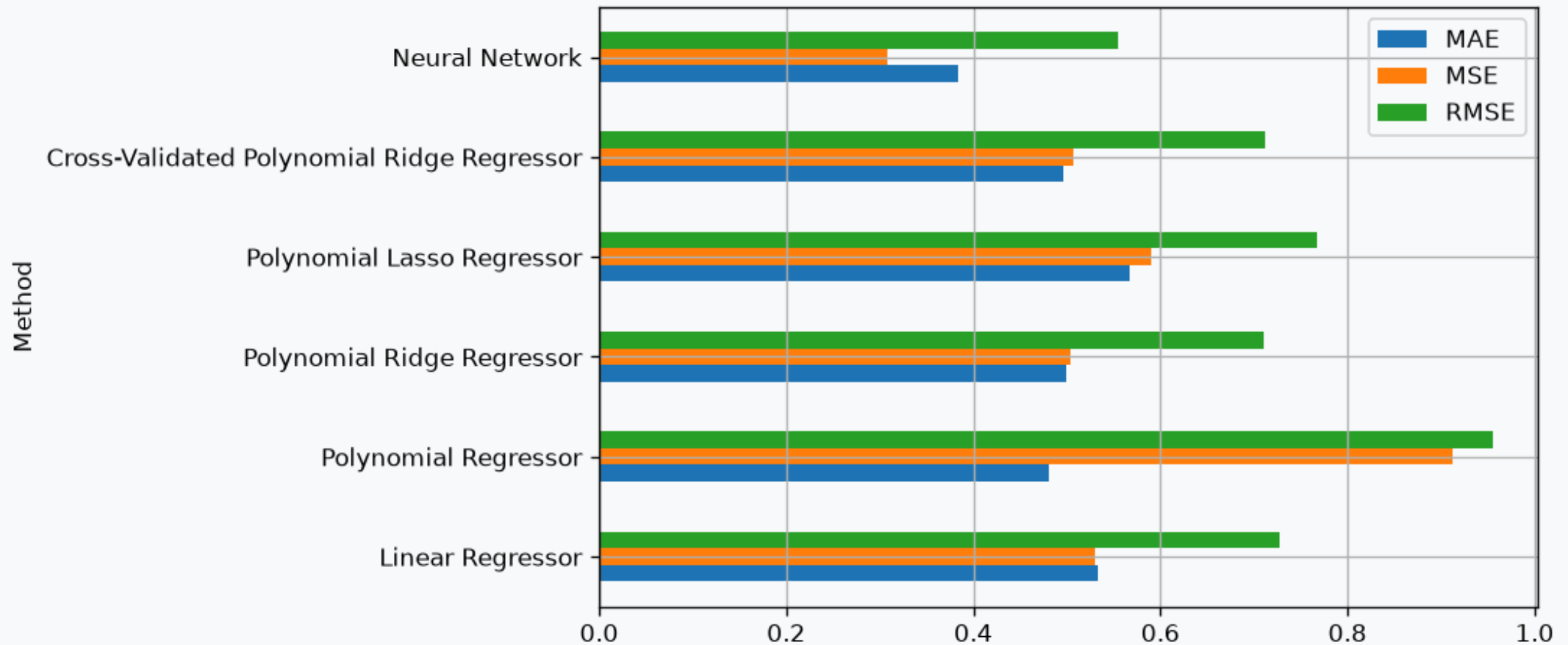
Every model we tried, on the same validation set:

Model	Parameters	Val. MAE	Val. RMSE
Linear regressor	—	0.533	0.728
Polynomial regressor	degree = 2	0.480	0.955
Polynomial ridge regressor	degree = 2, $\alpha = 300$	0.499	0.710
Polynomial lasso regressor	degree = 2, $\alpha = 0.04$	0.567	0.768
Cross-validated polynomial ridge	degree = 2, $\alpha = 250$	0.496	0.712
Neural network	default <code>MLPRegressor</code>	0.384	0.555

Thanks to the unified interface, a neural network is four lines away, and lands in the same table without our knowing how it works inside.
Knowing how it works is what makes it possible to tune it when it fails.



Comparison and Model Selection (cont.)



- The neural network wins on validation by a clear margin — 0.555 against 0.710 for the best regularised polynomial — so **it** is the model we take to the test set, where it scores **RMSE 0.549**. The regularised models are not wasted work: they are what tells us that 0.555 is a real gain and not the usual reward for extra capacity.

Only now, with one model chosen, do we touch the test set — once.



Conclusions and Next Steps



We Have Explored:

Metrics for prediction

Overfitting and the danger of capacity

Regularisation: ridge and lasso

λ as the bias-variance knob

The scikit-learn workflow

Grid search, pipelines and cross-validation

- In the next lectures, we will look at classification problems.

References

- Chapter 3 of Heumann, Christian, and Michael Schomaker Shalabh. *Introduction to statistics and data analysis*. Springer International Publishing Switzerland, 2016.
- Parts of chapter 6 of James, Gareth Gareth Michael. *An introduction to statistical learning: with applications in Python*, 2023.
<https://www.statlearning.com>
- Lecture notes: **Chapter 14 — Regression for prediction**

